DEEP LEARNING | SEQUENCE MODELS

From LSTM and GRU to Encoder–Decoder and Attention

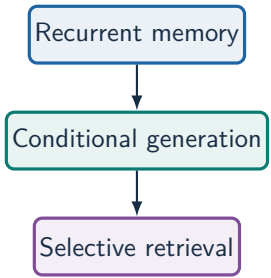
Architecture, motivation, equations and worked translation examples

Classroom lecture | IIT Bhilai | Based on CS7015 Lecture 16

The learning objectives

By the end you should be able to:

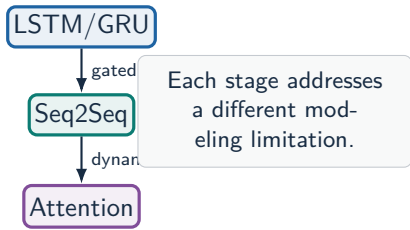
1. Explain what LSTM/GRU gates improve.
2. Draw a recurrent encoder–decoder.
3. Identify the fixed-context bottleneck.
4. Calculate alignment weights and c_t.
5. Trace one attentive decoding step.



Teaching sequence adapted from Khapra, CS7015 Deep Learning, Lecture 16.

From recurrent memory to selective retrieval

1. What LSTM and GRU solve—and do not solve
2. How encoder–decoder models generate sequences
3. Why fixed-length context becomes a bottleneck
4. How Bahdanau attention learns soft alignment
5. Computation, examples, and limitations



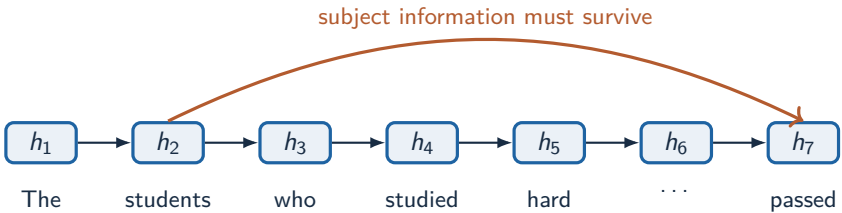
Basis: M. M. Khapra, CS7015 Lecture 16 (IIT Madras); architectural references: Cho et al. (2014), Sutskever et al. (2014), Bahdanau et al. (2015).

PART I

Why gated recurrent memory is not enough

Start with the sequential information path, then revisit what the LSTM and GRU actually fix.

A language task that needs a distant word



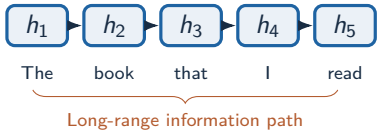
To predict agreement or meaning later, relevant evidence must be transported through multiple recurrent steps.

Illustrative English example; basic sequential dependency motivation from Lecture 16.

Sequence modeling: information must survive time

A recurrent network summarizes the current input and its past:

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b).$$

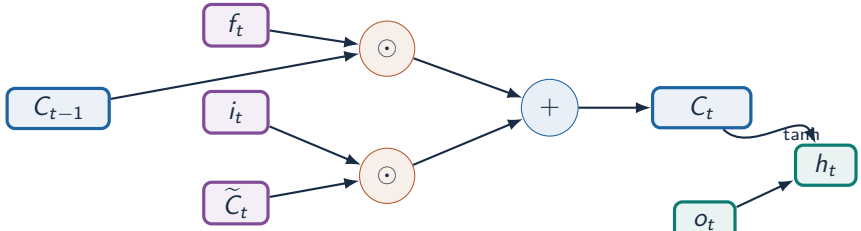


- h_t is a *fixed-dimensional* vector, although the input sequence may be arbitrarily long.
- To predict later tokens, the network must preserve earlier relevant information.
- Repeated transformations can make long-range dependencies hard to learn.

Key idea: A recurrent state provides memory, but not unlimited memory.

Elman-style recurrent formulation; motivation adapted from CS7015 Lecture 16.

Inside the LSTM: memory transport versus new content

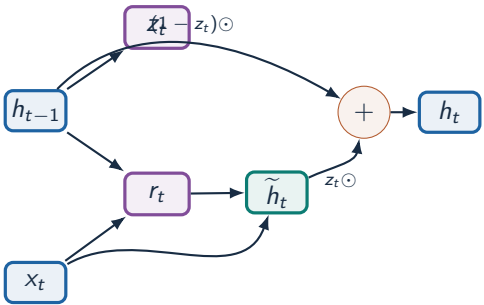


f_t: retain old memory
i_t: write candidate
o_t: expose output

$$C_t = f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t, \qquad h_t = o_t \odot \tanh(C_t)$$

LSTM equations: Hochreiter and Schmidhuber (1997); simplified dataflow reconstructed for teaching.

Inside the GRU: update and reset



Interpretation

- Reset gate r_t: controls use of old state when forming a candidate.
- Update gate z_t: mixes candidate with old state.
- No separate C_t memory channel.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Cho et al. (2014); one of two common update-gate parameterizations.

Why LSTM and GRU improve ordinary RNNs

LSTM: dedicated cell state C_t and three gates:

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}; x_t] + b_f) \\
i_t &= \sigma(W_i[h_{t-1}; x_t] + b_i) \\
\widetilde{C}_t &= \tanh(W_c[h_{t-1}; x_t] + b_c) \\
C_t &= f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t \\
o_t &= \sigma(W_o[h_{t-1}; x_t] + b_o) \\
h_t &= o_t \odot \tanh(C_t)
\end{aligned}
$$

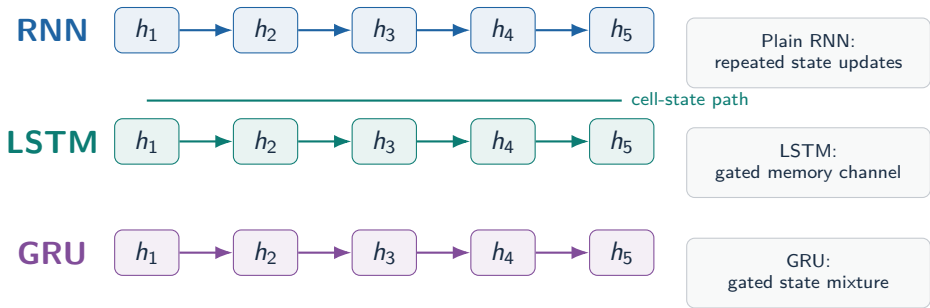
GRU: no separate cell state; two gates:

$$
\begin{aligned}
z_t &= \sigma(W_z[x_t; h_{t-1}] + b_z) \\
r_t &= \sigma(W_r[x_t; h_{t-1}] + b_r) \\
\widetilde{h}_t &= \tanh(W_h[x_t; r_t \odot h_{t-1}] + b_h) \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t
\end{aligned}
$$



LSTM: Hochreiter & Schmidhuber (1997); GRU: Cho et al. (2014). GRU uses the update-gate convention shown here; alternative notation swaps z_t and $(1 - z_t)$.

Compare memory mechanisms side by side

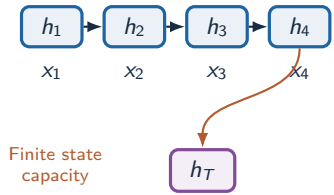


Conceptual unfolded recurrent schematics; all three process sequential states; gating does not create source-position retrieval.

What gating fixes—and what it does not

Gating helps:

- Learn when information should be stored, forgotten, or exposed.
- Create more direct paths for information and gradients.
- Model longer dependencies than a basic RNN in many settings.



But gating does not guarantee:

- Perfect retention of all past tokens.
- Unlimited representation capacity.
- Direct access to every earlier source position during decoding.

Key idea: Gating improves memory; it does not provide direct access to every source state.

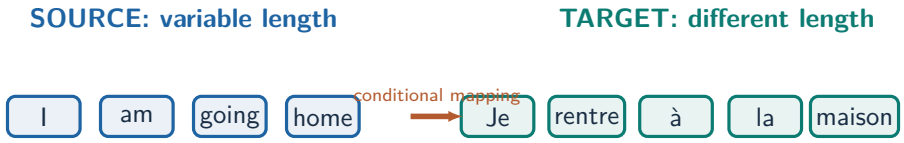
Clarification beyond the source lecture: gating alleviates optimization issues but does not mathematically guarantee complete information retention.

PART II

Encoder–decoder sequence learning

Two recurrent networks learn a mapping between a source sequence and a target sequence, even when their lengths differ.

Why not make translation one ordinary RNN?



The decoder needs a learned source representation while generating output one token at a time. Input length T and output length U need not match.

Conditional sequence modeling in Cho et al. (2014) and Sutskever et al. (2014). Translation example is illustrative.

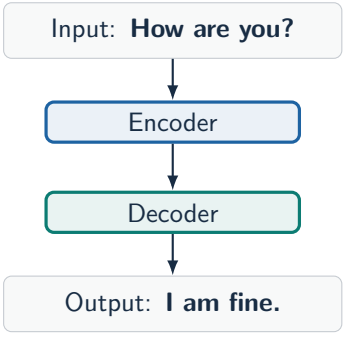
From next-token prediction to conditional generation

Language modeling: predict the next token from previous tokens.

$$P(y_t \mid y_1, \ldots, y_{t-1}).$$

Sequence-to-sequence: additionally condition on an input sequence $x = (x_1, \ldots, x_T)$.

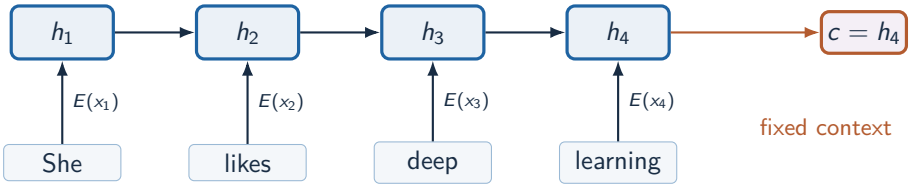
$$P(y \mid x) = \prod_{t=1}^{U} P(y_t \mid y_{<t}, x).$$



- Input and output can have *different lengths*.
- Typical tasks: translation, dialogue, summarization.

Example and setup adapted from CS7015 Lecture 16, modules 16.1–16.2.

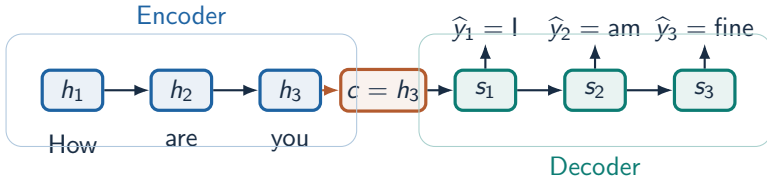
Trace the encoder: one source token per time step



$$h_i = \mathrm{RNN}_{enc}(E(x_i), h_{i-1}), \quad c = h_T$$

Unidirectional fixed-context recurrent encoder; reconstruction based on classical sequence-to-sequence papers.

The classical encoder–decoder architecture



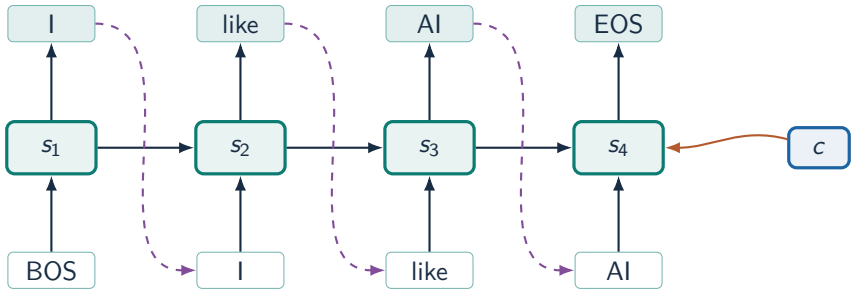
Encoder: $h_i = f_{\text{enc}}(h_{i-1}, x_i)$; summarize the source using $c = h_T$.

Decoder: initialize s_0 from c, then generate one target token at a time.

Key idea: In a classic fixed-context model, the decoder receives the input primarily through c.

Architecture redrawn from Sutskever, Vinyals & Le (2014); Cho et al. (2014); CS7015 Lecture 16.

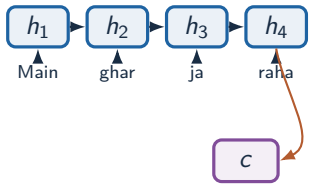
Trace the decoder: prediction becomes the next input



Simplified autoregressive decoder; dashed links illustrate feedback during inference, not separate trainable modules.

Encoder computation: words become source annotations

$$e_i = E_{\text{src}}(x_i)$$
$$h_i = f_{\text{enc}}(h_{i-1}, e_i), \quad i = 1, \ldots, T$$
$$c = h_T \quad \text{(unidirectional fixed-context model)}$$

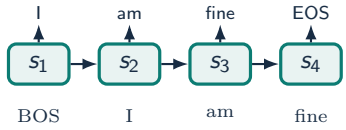


- E_{src} maps token IDs to dense embeddings.
- h_i summarizes the prefix $x_1, \ldots, x_i$.
- f_{enc} may be an LSTM or GRU cell.

The Hindi-to-English motivation is taken from the source lecture; diagram and notation are newly redrawn.

Decoder computation and the start/stop tokens

$$
\begin{aligned}
s_0 &= g(c) \\
s_t &= f_{\mathrm{dec}}(s_{t-1}, E_{\mathrm{tgt}}(y_{t-1})) \\
p_t &= \mathrm{softmax}(W s_t + b) \\
\widehat{y}_t &= \arg\max_{v \in \mathcal{V}} p_t(v)
\end{aligned}
$$



- At $t = 1$, the previous token is $\langle \mathrm{BOS} \rangle$.
- Generation ends at $\langle \mathrm{EOS} \rangle$ or a maximum length.
- Each step depends on earlier generated tokens and encoder-derived initialization.

CS7015 Lecture 16 decoder construction; formal symbols standardized to s_t, c, and BOS/EOS.

Training and inference are not the same rollout

TRAINING



Teacher forcing: ground truth previous output

INFERENCE



Autoregressive: model's own previous output

Training/inference distinction for autoregressive recurrent decoder; teacher forcing is standard supervised sequence learning practice.

Training versus inference: teacher forcing

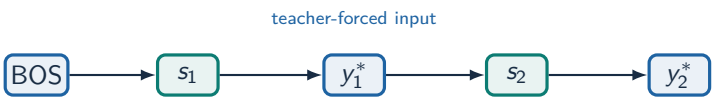
Training

- Supply the *ground-truth* previous token to the decoder (teacher forcing).
- Minimize negative log-likelihood:

$$\mathcal{L}(\theta) = -\sum_{t=1}^{U} \log P_\theta(y_t^* \mid y_{<t}^*, x).$$

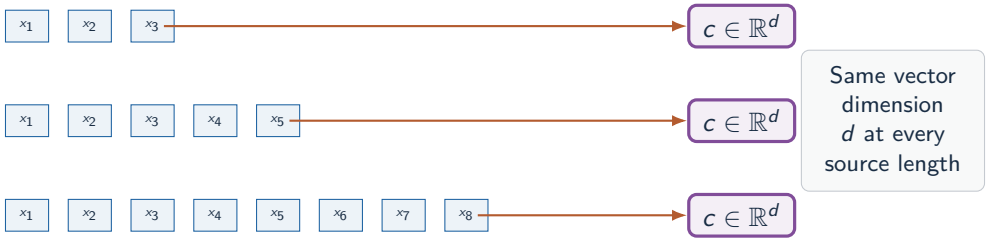
Inference

- Ground-truth target tokens are unavailable.
- Feed the model's own previous predictions, using greedy search or beam search.
- Prediction errors can propagate to later steps (exposure bias).



Training objective follows CS7015 Lecture 16; teacher forcing / exposure-bias terminology is supplemental explanation.

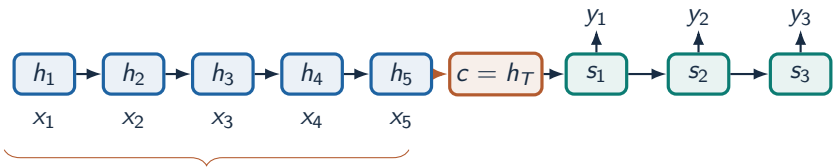
Thought experiment: sentence gets longer, context does not



Question for students: if a rare detail occurs early in a long sentence, how does the decoder retrieve it?

Fixed-dimensional final encoder state in the original non-attentive interface; illustrative, not an impossibility theorem.

The fixed-context bottleneck



- c has constant dimensionality even when the source length T increases.
- Later outputs have no separate, direct lookup of earlier encoder states.
- This is a *representational and optimization bottleneck*, especially for long or detailed inputs.

Fixed-length vector critique: Bahdanau, Cho & Bengio (2015); CS7015 Lecture 16, introduction to attention.

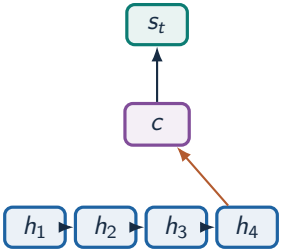
PART III

Attention: dynamic source retrieval

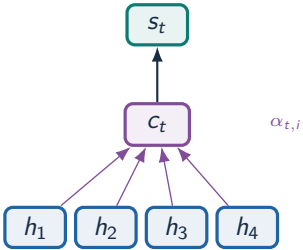
Instead of reading one final summary, the decoder forms a fresh weighted summary of all encoder annotations at each output step.

The big visual change: one vector versus many connections

Before: fixed context



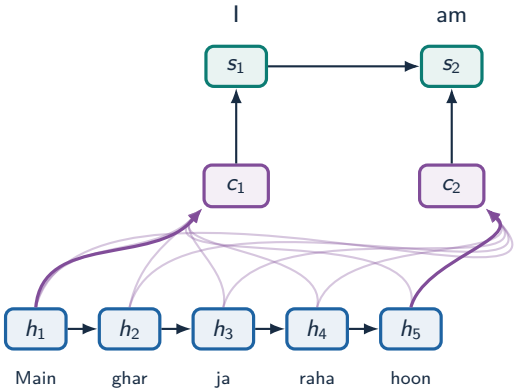
After: step-dependent context



Key idea: Attention opens a source-position pathway that can vary with the next target token.

Architectural difference based on Bahdanau, Cho and Bengio (2015).

The solution: a context vector for every output step

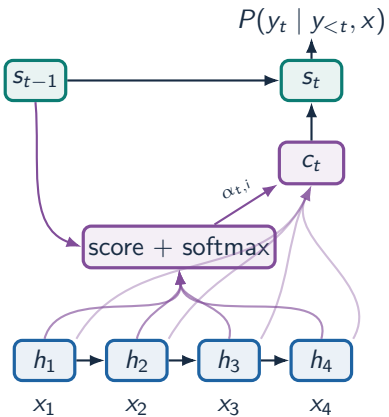


$c_t = \sum_i \alpha_{t,i} h_i$

- The decoder retains recurrent state s_{t-1}, *and* gains direct weighted access to all encoder states h_i.
- Weights $\alpha_{t,i}$ depend on the current decoding step t.

Bahdanau, Cho & Bengio (2015), attentive encoder–decoder; Hindi-English example from CS7015 Lecture 16.

Bahdanau attention: original research architecture

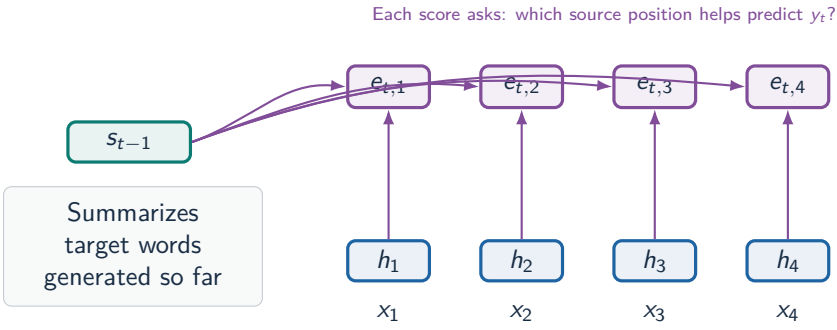


How to read it

1. Use previous decoder state as a *query*.
2. Score each encoder annotation.
3. Normalize scores.
4. Form weighted context c_t.
5. Generate the next output.

Research-style additive-attention diagram reconstructed in TikZ from Bahdanau et al., "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR 2015. Not a reproduction of the original figure.

Why the previous decoder state is the query

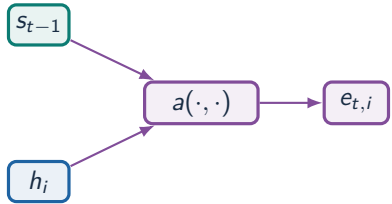


Additive score $e_{t,i} = a(s_{t-1}, h_i)$ from Bahdanau et al. (2015).

Step 1: score each source annotation

Additive (Bahdanau) scoring function:

$$e_{t,i} = v_a^\top \tanh(W_a s_{t-1} + U_a h_i + b_a).$$



- s_{t-1} describes what the decoder has generated so far.
- h_i represents source position i.
- $e_{t,i}$ is a *scalar compatibility score*, not yet a probability.
- The matrices and vector are trainable parameters.

Key idea: Attention first learns *where the decoder should look*.

Bahdanau et al. (2015): learned alignment model $a(s_{t-1}, h_i)$. Bias b_a is a common implementation extension.

Numerical practice: convert scores to attention weights

Suppose the scores for three source positions are

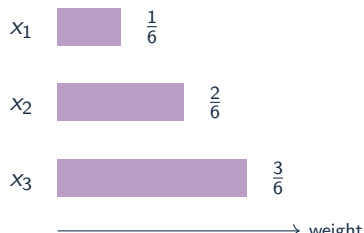
$$e_t = [0,\ \ln 2,\ \ln 3].$$

Exponentiate each score:

$$\exp(e_t) = [1, 2, 3].$$

Normalize by the sum $1 + 2 + 3 = 6$:

$$\boxed{\alpha_t = \left[\frac{1}{6}, \frac{2}{6}, \frac{3}{6}\right]}.$$



Key idea: Softmax converts arbitrary real-valued scores into nonnegative weights summing to one.

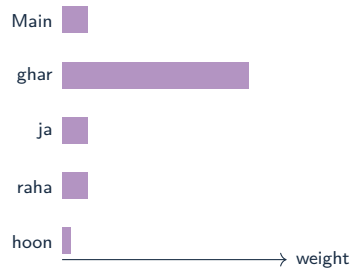
Illustrative exact calculation; no trained model values assumed.

Step 2: normalize scores using softmax

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}, \qquad \alpha_{t,i} \geq 0, \qquad \sum_i \alpha_{t,i} = 1.$$

Source token	$e_{t,i}$	$\alpha_{t,i}$
Main	0	0.093
ghar	2	0.687
ja	0	0.093
raha	0	0.093
hoon	-1	0.034

Illustrative rounded values (not model measurements).



Softmax definition from Bahdanau et al. (2015). Example is pedagogical; weights should be computed exactly when used numerically.

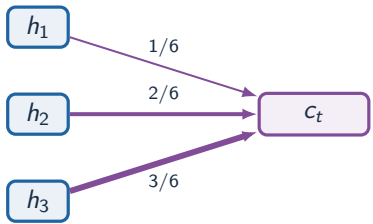
Numerical practice: construct c_t as a weighted sum

$$h_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad h_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad h_3 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

$$\alpha_t = \left[\frac{1}{6}, \frac{2}{6}, \frac{3}{6}\right]$$

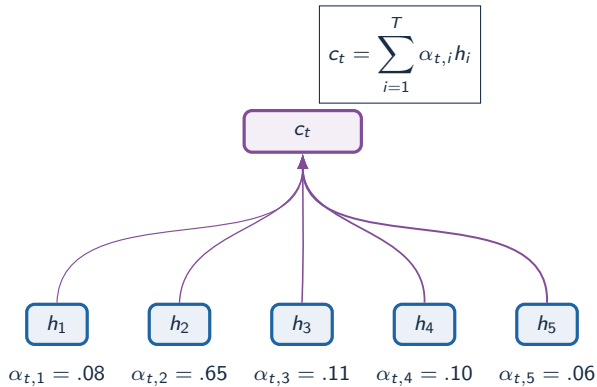
$$c_t = \frac{1}{6}h_1 + \frac{2}{6}h_2 + \frac{3}{6}h_3$$

$$= \boxed{\begin{bmatrix} \frac{7}{6} \\ \frac{4}{3} \end{bmatrix}}$$



Two-dimensional numerical toy example; illustrates elementwise vector averaging under normalized weights.

Step 3: create the dynamic context vector

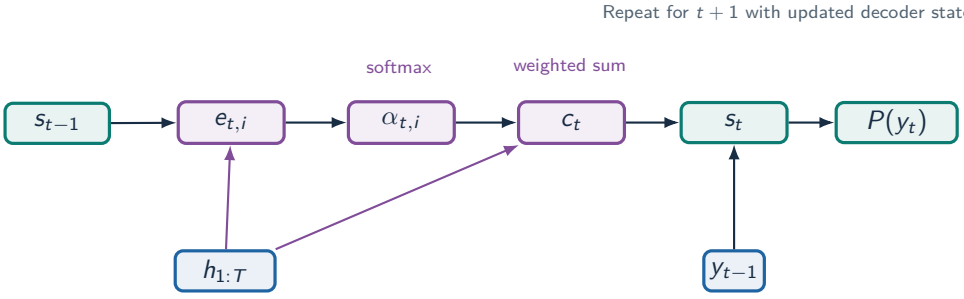


Weighted sum
of encoder states

- c_t has the same dimension as the encoder annotations h_i.
- Large $\alpha_{t,i}$ values give greater influence to selected source positions.
- Crucially, c_t *changes with* t; it is not necessarily equal to h_T.

Context-vector definition: Bahdanau et al. (2015), also explained in CS7015 Lecture 16, module 16.3.

One complete attentive decoding step



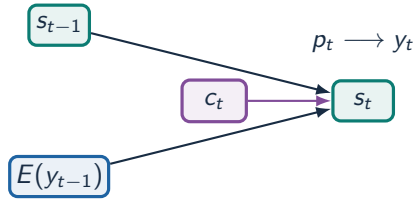
$$e_{t,i} \rightarrow \alpha_{t,i} \rightarrow c_t \rightarrow s_t \rightarrow P(y_t)$$

Unrolled computation of recurrent additive attention; Bahdanau et al. (2015).

Step 4: decoder combines history and source context

A representative attentive decoder:

$$s_t = f_{\text{dec}}(s_{t-1}, E(y_{t-1}), c_t)$$
$$p_t = \text{softmax}\big(W_o[s_t; c_t] + b_o\big)$$
$$\widehat{y}_t = \arg\max_v p_t(v)$$



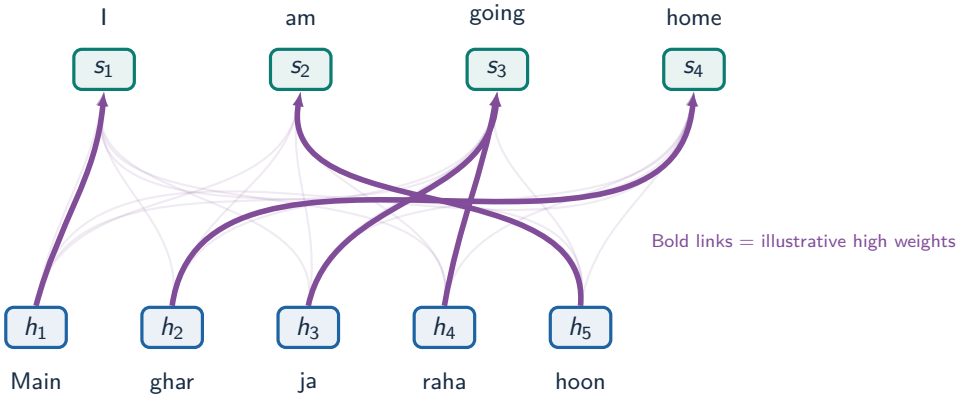
Three distinct inputs:

1. Previous decoder state s_{t-1}
2. Previous output token y_{t-1}
3. Current, learned source context c_t

Key idea: Attention augments recurrent decoding; it does not remove the recurrence in the original Bahdanau architecture.

Representative conditional recurrent decoder consistent with Bahdanau et al. (2015); exact output-layer parameterization varies by implementation.

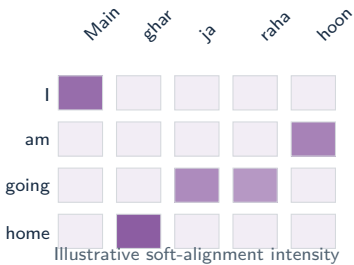
Attention changes from target word to target word



Pedagogical qualitative alignment illustration, not actual measured attention. Original alignment visualizations: Bahdanau et al. (2015).

Alignment example: source tokens versus outputs

Source: Main ghar ja raha hoon
Target: I am going home



Observe:

- "home" can strongly attend to "ghar."
- "going" may use the multiword predicate "ja raha."
- Alignment is *soft*, not a compulsory one-to-one matching.
- Attention can reorganize source information when target word order differs.

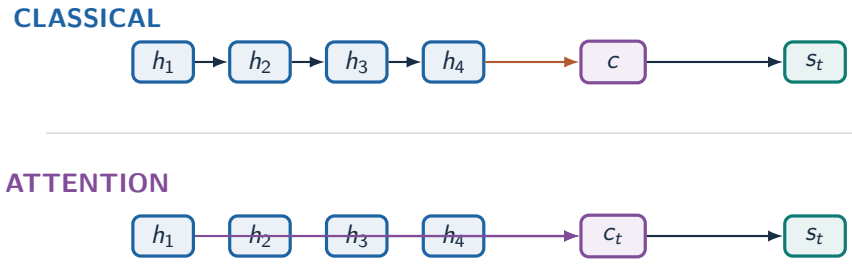
Example task from CS7015 Lecture 16. Heatmap created for teaching, not extracted from a trained model. See Bahdanau et al. (2015) for learned alignment visualizations.

PART IV

Comparison, limits and assessment

Consolidate the full computational graph and test whether students can explain the motivation and mathematics.

The same decoder: without attention and with attention



Reconstruction showing alternative encoder–decoder interfaces from Cho et al. (2014) and Bahdanau et al. (2015).

Fixed-context Seq2Seq versus attention-based Seq2Seq

Aspect	Fixed-context model	Attention-based model
Source interface	One context, often $c = h_T$	All encoder annotations $h_1, \ldots, h_T$
Context during decoding	Primarily fixed	$c_t = \sum_i \alpha_{t,i} h_i$ varies by step
Alignment	Implicit in learned state	Explicit soft scores and weights
Long-source handling	Stronger compression bottleneck	Easier retrieval of relevant positions
Additional cost	Compact source interface	Attention scoring across source positions
Remaining limitations	State compression	Alignment may be diffuse; cost grows with lengths

Key idea: Attention adds content-dependent retrieval; it does not promise perfect reasoning or unlimited context.

Comparison based on Sutskever et al. (2014), Bahdanau et al. (2015), and CS7015 Lecture 16.

Quick classroom checkpoint (answer before revealing)

1. Can a GRU itself access arbitrary encoder states h_i directly?
2. If attention scores are $[0, 0, 0]$, what are the weights?
3. Is c_t always identical to the encoder's final state h_T?
4. At time t, what makes attention weights different from time $t - 1$?
5. Which model uses s_{t-1} and h_i to calculate alignment?

Understanding questions anchored in classical recurrent additive attention.

Checkpoint: explained answers

1. **No.** A plain GRU cell updates its own recurrent state; cross-source retrieval requires added architecture.
2. $[1/3, 1/3, 1/3]$**.** Softmax of equal scores is uniform.
3. **No.** In attention, $c_t = \sum_i \alpha_{t,i} h_i$ changes with the decoder state.
4. **The query changes.** Scores depend on evolving decoder state s_{t-1}.
5. **Bahdanau additive attention.** It uses learned compatibility $a(s_{t-1}, h_i)$.

Solutions use equations defined earlier in the deck.

Important distinctions and common misconceptions

1. **"LSTM solves long sequences completely."** No. Gating often helps retention and gradient flow, but finite state capacity and optimization challenges remain.
2. **"Encoder–decoder is the same as attention."** No. Seq2Seq existed before attention; attention changes the source-to-decoder interface.
3. **"Attention selects exactly one word."** No. Softmax creates a distribution across all source positions.
4. **"Attention means the Transformer."** No. Bahdanau attention operated with recurrent encoder and decoder; Transformer self-attention is a later architectural development.
5. **"A large weight proves human-like explanation."** Not necessarily. Attention weights are learned model quantities, not guaranteed faithful explanations.

First four distinctions follow original RNN/Seq2Seq/attention research. Interpretability caveat is a contemporary caution, not a claim made by the source lecture.

Key research papers and what each contributed

Paper	Main architecture	Lecture takeaway
Hochreiter & Schmidhuber (1997)	Long Short-Term Memory	Gated path for persistent memory.
Cho et al. (2014)	RNN encoder–decoder; GRU	Map source sequence to conditional target sequence.
Sutskever et al. (2014)	Multilayer LSTM Seq2Seq	End-to-end recurrent sequence learning with fixed-dimensional source code.
Bahdanau et al. (2015)	Joint alignment and translation	Compute a source-aware context separately for each decoder output.
Vaswani et al. (2017)	Transformer	Later development: attention-based sequence modeling without recurrent layers.

Source lecture: Mitesh M. Khapra, IIT Madras, CS7015 Lecture 16. Papers are cited by their publication year; original attention paper was posted in 2014 and published at ICLR 2015.

End-to-end recap and questions for students

Complete attentive Seq2Seq computation

$$
\begin{aligned}
h_i &= f_{\text{enc}}(h_{i-1}, x_i) \\
e_{t,i} &= a(s_{t-1}, h_i) \\
\alpha_{t,i} &= \text{softmax}_i(e_{t,i}) \\
c_t &= \sum_i \alpha_{t,i} h_i \\
s_t &= f_{\text{dec}}(s_{t-1}, y_{t-1}, c_t) \\
p_t &= P(y_t \mid y_{<t}, x)
\end{aligned}
$$

Check understanding

1. What information is held in h_i and s_t?
2. Why does fixed context become a bottleneck?
3. Why do we apply softmax to $e_{t,i}$?
4. Can multiple source positions matter for one output?
5. Does adding attention eliminate LSTM/GRU?

Key idea: Gated memory $\rightarrow$ conditional Seq2Seq $\rightarrow$ dynamic, learned alignment.

Lecture basis: Khapra, CS7015 Lecture 16. Papers: Cho et al. (2014), Sutskever et al. (2014), Bahdanau et al. (2015).

One idea to remember

Memory carries information forward.
Attention lets the decoder look
back at relevant source states.

Next lecture: attention variants, limitations and Transformers